```
!pip install statsmodels
```

```
Requirement already satisfied: statsmodels in c:\users\hp user\anaconda3\new folder\anaconda\lib\site-packages (0.14.5)
Requirement already satisfied: numpy<3,>=1.22.3 in c:\users\hp user\anaconda3\new folder\anaconda\lib\site-packages (from statsm
Requirement already satisfied: scipy!=1.9.2,>=1.8 in c:\users\hp user\anaconda3\new folder\anaconda\lib\site-packages (from stat
Requirement already satisfied: pandas!=2.1.0,>=1.4 in c:\users\hp user\anaconda3\new folder\anaconda\lib\site-packages (from sta
Requirement already satisfied: patsy>=0.5.6 in c:\users\hp user\anaconda3\new folder\anaconda\lib\site-packages (from statsmodel
Requirement already satisfied: packaging>=21.3 in c:\users\hp user\anaconda3\new folder\anaconda\lib\site-packages (from statsmo
Requirement already satisfied: python-dateutil>=2.8.2 in c:\users\hp user\anaconda3\new folder\anaconda\lib\site-packages (from
Requirement already satisfied: pytz>=2020.1 in c:\users\hp user\anaconda3\new folder\anaconda\lib\site-packages (from pandas!=2.
Requirement already satisfied: tzdata>=2022.7 in c:\users\hp user\anaconda3\new folder\anaconda\lib\site-packages (from pandas!=
Requirement already satisfied: six>=1.5 in c:\users\hp user\anaconda3\new folder\anaconda\lib\site-packages (from python-dateuti
```

```
import pandas as pd
import numpy as np
import matplotlib.pyplot as plt
import seaborn as sns
import statsmodels.api as sm
from statsmodels.stats.outliers_influence import variance_inflation_factor
```

```
#Loading Dataset#
df = pd.read_csv('marketing_sales_data.csv')
```

```
print(df.head())
```

```
       TV      Radio  Social Media Influencer       Sales
0     Low   3.518070      2.293790      Micro   55.261284
1     Low   7.756876      2.572287       Mega   67.574904
2    High  20.348988      1.227180      Micro  272.250108
3  Medium  20.108487      2.728374       Mega  195.102176
4    High  31.653200      7.776978       Nano  273.960377
```

```
#loading dataset information#
print(df.info())
```

```
<class 'pandas.core.frame.DataFrame'>
RangeIndex: 572 entries, 0 to 571
Data columns (total 5 columns):
 #   Column        Non-Null Count  Dtype
---  ------        --------------  -----
 0   TV            572 non-null    object
 1   Radio         572 non-null    float64
 2   Social Media  572 non-null    float64
 3   Influencer    572 non-null    object
 4   Sales         572 non-null    float64
dtypes: float64(3), object(2)
memory usage: 22.5+ KB
None
```

```
#Checking missing values#
print(df.isnull().sum())
```

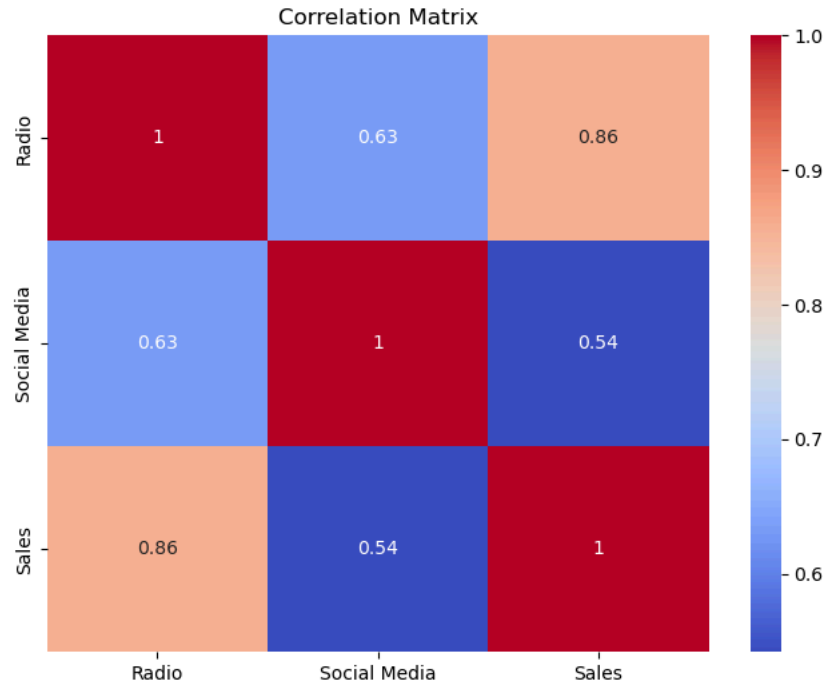
```
TV              0
Radio           0
Social Media    0
Influencer      0
Sales           0
dtype: int64
```

```
#Descriptive statistics#
print(df.describe())
```

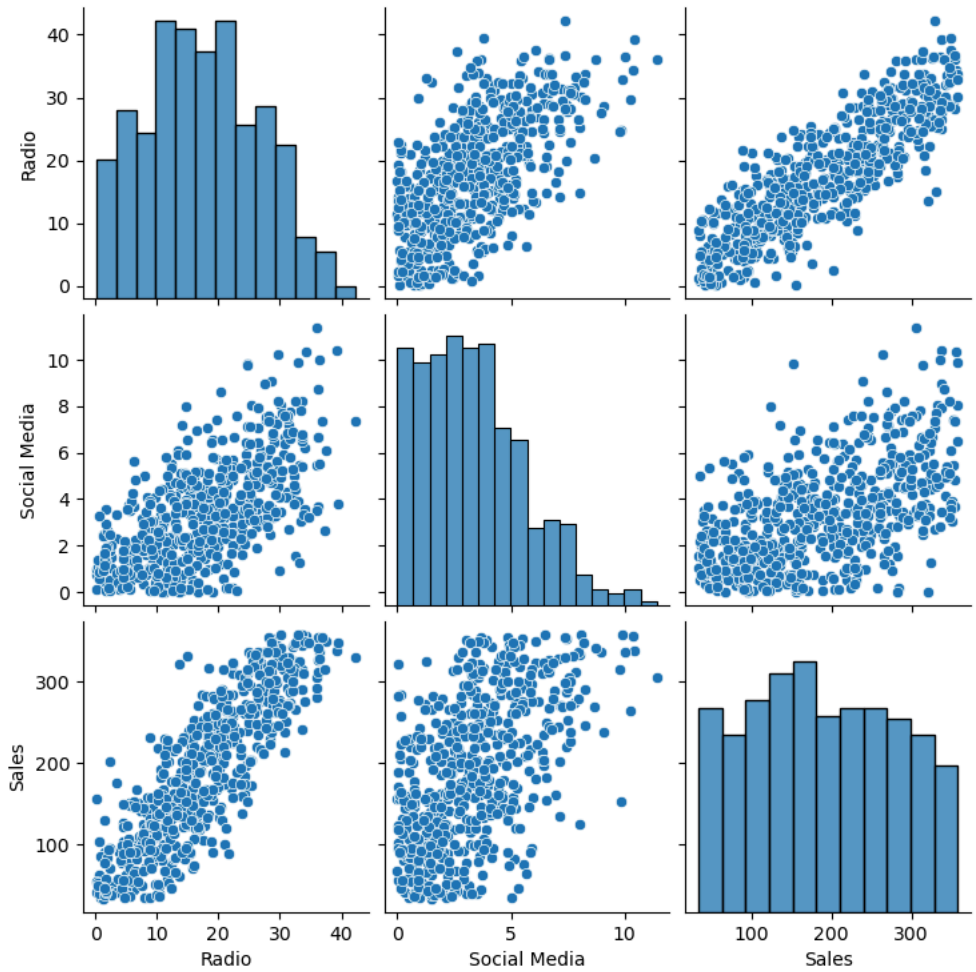
```
            Radio  Social Media       Sales
count  572.000000    572.000000  572.000000
mean    17.520616      3.333803  189.296908
std      9.290933      2.238378   89.871581
min      0.109106      0.000031   33.509810
25%     10.699556      1.585549  118.718722
50%     17.149517      3.150111  184.005362
75%     24.606396      4.730408  264.500118
max     42.271579     11.403625  357.788195
```

```
#EXPLORATORY DATA ANALYSIS#
#Correlation Matrix#
numeric_df = df.select_dtypes(include=np.number)

plt.figure(figsize=(8,6))
sns.heatmap(numeric_df.corr(),
            annot=True,
            cmap='coolwarm')
plt.title('Correlation Matrix')
plt.show()
```




```
#Pairplot#
sns.pairplot(df)
plt.show()
```




```
#DATA PREPROCESSING#
# Converting Categorical Variables#
df_encoded = pd.get_dummies(
    df,
    columns=['TV','Influencer'],
    drop_first=True
)
print(df_encoded.head())
```

```
       Radio  Social Media       Sales  TV_Low  TV_Medium  Influencer_Mega  \
0   3.518070      2.293790   55.261284    True      False            False
1   7.756876      2.572287   67.574904    True      False             True
2  20.348988      1.227180  272.250108   False      False            False
3  20.108487      2.728374  195.102176   False       True             True
4  31.653200      7.776978  273.960377   False      False            False

   Influencer_Micro  Influencer_Nano
0              True            False
1             False            False
2              True            False
3             False            False
4             False             True
```

```
print(X.dtypes)
```

```
Radio               float64
Social Media        float64
TV_Low                 bool
TV_Medium              bool
Influencer_Mega        bool
Influencer_Micro       bool
Influencer_Nano        bool
dtype: object
```

```
#Converting all columns to numeric float#
X = X.astype(float)
```

```
df_encoded = pd.get_dummies(
    df,
    columns=['TV', 'Influencer'],
    drop_first=True,
    dtype=int
)
X = df_encoded.drop('Sales',axis=1)
#Adding constant#
X_const = sm.add_constant(X)
vif_data = pd.DataFrame()
vif_data["Variable"] = X_const.columns
vif_data["VIF"] =[
    variance_inflation_factor(X_const.values, i)
    for i in range(X_const.shape[1])
]
print("\nVariance Inflation Factor (VIF)")
print(vif_data)
```

```
Variance Inflation Factor (VIF)
           Variable        VIF
0             const  31.649565
1             Radio   3.479641
2      Social Media   1.669098
3            TV_Low   4.076384
4         TV_Medium   2.233350
5   Influencer_Mega   1.593889
6  Influencer_Micro   1.618434
7   Influencer_Nano   1.627430
```

```
# Building Regression Model#
y = df_encoded['Sales']
model = sm.OLS(y, X_const).fit()
print(model.summary())
```

```
                            OLS Regression Results
==============================================================================
Dep. Variable:                  Sales   R-squared:                       0.904
Model:                            OLS   Adj. R-squared:                  0.903
Method:                 Least Squares   F-statistic:                     760.4
Date:                Tue, 23 Jun 2026   Prob (F-statistic):          1.82e-282
Time:                        02:36:04   Log-Likelihood:                -2713.4
No. Observations:                 572   AIC:                             5443.
Df Residuals:                     564   BIC:                             5478.
Df Model:                           7
Covariance Type:            nonrobust
==============================================================================================
                     coef    std err          t      P>|t|      [0.025      0.975]
----------------------------------------------------------------------------------------------
const              217.4784      6.584     33.031      0.000     204.546     230.411
Radio                2.9735      0.235     12.644      0.000       2.512       3.435
Social Media        -0.1391      0.676     -0.206      0.837      -1.467       1.189
TV_Low            -154.5736      4.949    -31.231      0.000    -164.295    -144.852
TV_Medium          -75.5947      3.647    -20.726      0.000     -82.759     -68.431
Influencer_Mega      2.4948      3.462      0.721      0.471      -4.305       9.295
Influencer_Micro     2.9391      3.378      0.870      0.385      -3.695       9.574
Influencer_Nano      0.8015      3.346      0.240      0.811      -5.770       7.373
==============================================================================
Omnibus:                       58.711   Durbin-Watson:                   1.874
Prob(Omnibus):                  0.000   Jarque-Bera (JB):               17.802
Skew:                           0.057   Prob(JB):                     0.000136
Kurtosis:                       2.143   Cond. No.                         147.
==============================================================================

Notes:
[1] Standard Errors assume that the covariance matrix of the errors is correctly specified.
```

```
# Adjusting R-Squared#
print("Adjusted R-Squared:",
     model.rsquared_adj)
```

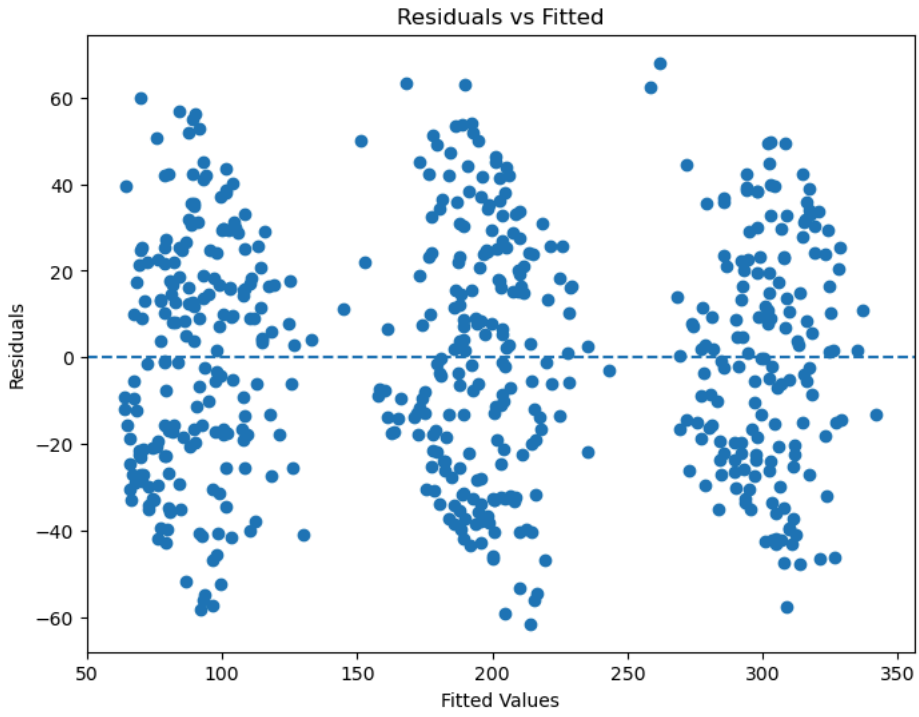
```
Adjusted R-Squared: 0.9030011006551627
```

```
#Coefficient Interpretation#
coef_df = pd.DataFrame({
    'Variable': model.params.index,
    'Coefficient': model.params.values,
    'P-Value': model.pvalues.values
})
print(coef_df)
```

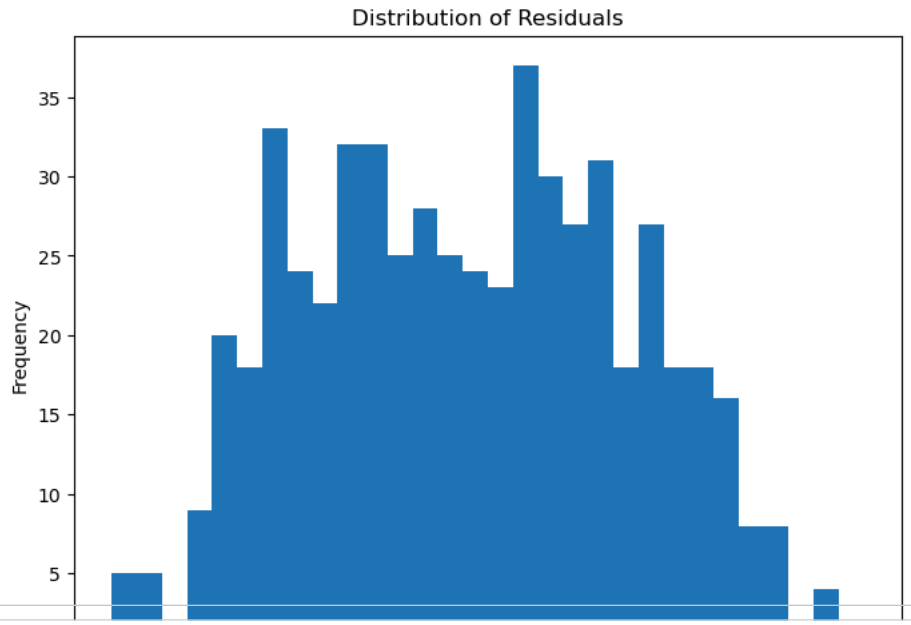
```
           Variable  Coefficient        P-Value
0             const   217.478405  5.906914e-134
1             Radio     2.973532   1.943682e-32
2      Social Media    -0.139054   8.371166e-01
3            TV_Low  -154.573626  4.536854e-125
4         TV_Medium   -75.594662   2.277532e-71
5   Influencer_Mega     2.494795   4.714413e-01
6  Influencer_Micro     2.939094   3.845913e-01
7   Influencer_Nano     0.801541   8.107454e-01
```

```
# Diagnostic Plots#
residuals = model.resid
fitted = model.fittedvalues
```

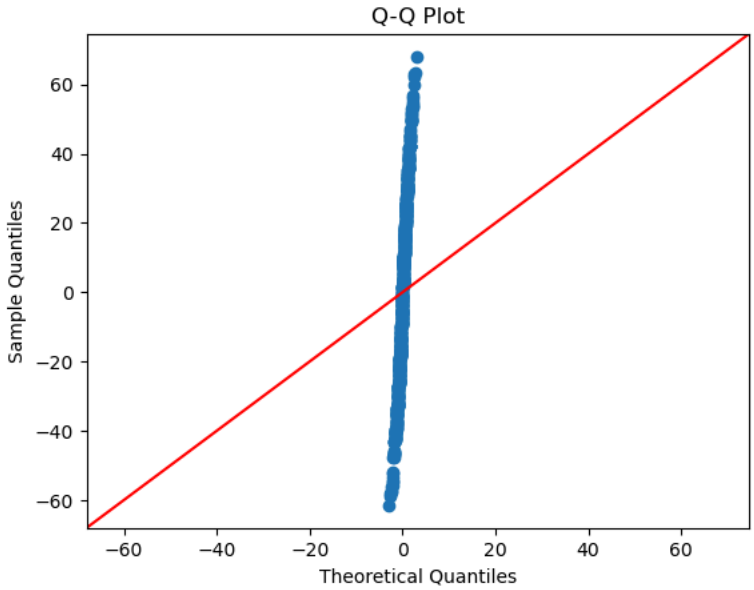
```
#Residuals Plot#
plt.figure(figsize=(8,6))
plt.scatter(fitted, residuals)
plt.axhline(y=0, linestyle='--')
plt.xlabel("Fitted Values")
plt.ylabel("Residuals")
plt.title("Residuals vs Fitted")
plt.show()
```




```
#Histogram#
plt.figure(figsize=(8,6))
plt.hist(residuals, bins=30)
plt.title("Distribution of Residuals")
plt.xlabel("Residual")
plt.ylabel("Frequency")
plt.show()
```




```
#Q-Q Plot#
sm.qqplot(residuals, line='45')
plt.title("Q-Q Plot")
plt.show()
```




```
#BUSINESS INTERPRETATION#
print("""BUSINESS INSIGHTS
1. Radio advertising has a positive and significant effect on Sales.
2.Higher TV advertising levels generates significantly higher sales compared with lower TV spending levels.
3.Social Media advertising has little impact on Sales.
4.Influencer category does not significantly affect Sales.
5.The model explains approximately 90% of variation in Sales, indicating excellent predictive performance.
```